

计算机基础

Violet

2026-04-11

Chapter 0

目 录

I	版本管理与开发工具	
1.	Git 版本控制详解	6
1.1.	版本控制系统概述	6
1.2.	Git 核心概念	7
1.3.	Git 基本操作	9
1.4.	分支管理	13
1.5.	远程协作	16
1.6.	高级操作	17
1.7.	标签管理	19
1.8.	Git 工作流	20
1.9.	.gitignore 配置	22
1.10.	Git 钩子 (Hooks)	24
1.11.	常见问题与解决方案	25
1.12.	Git 性能优化	27
1.13.	Git 最佳实践总结	28
II	数据库系统	
2.	数据库基础	31
III	并发编程与分布式系统	
3.	并发编程基础	33
4.	分布式系统理论	34
4.1.	分布式系统概述与挑战	34
4.2.	CAP 定理详解	35
4.3.	CAP 权衡与实际应用	38
4.4.	BASE 理论	40
4.5.	一致性模型	43
4.6.	分布式事务	45
4.7.	分布式锁实现	50
4.8.	Leader 选举算法	54
4.9.	拜占庭将军问题	56

4.10. FL 不可能定理	58
4.11. 总结	60



版本管理与开发工具

Chapter 1

Git 版本控制详解

Git 是目前最流行的分布式版本控制系统，由 Linus Torvalds 于 2005 年创建，用于 Linux 内核开发。它以其速度、数据完整性和对非线性工作流的强大支持而闻名。

NOTE

Git 的核心设计理念是**快照**而非**差异**。每次提交时，Git 都会存储整个项目的快照，如果文件没有变化，则只存储指向之前版本的链接。

1 版本控制系统概述

1.1 集中式 vs 分布式

集中式版本控制 (CVCS)

代表工具：SVN、CVS、Perforce

特点：

- 单一中央服务器存储所有版本历史
- 客户端只获取最新版本
- 必须联网才能进行大部分操作
- 中央服务器是单点故障

优点：

- 权限管理简单
- 学习曲线较低
- 适合小型团队

缺点：

- 网络依赖性强
- 中央服务器故障影响所有人
- 分支和合并操作较慢

分布式版本控制 (DVCS)

代表工具：Git、Mercurial、Bazaar

特点：

- 每个客户端都有完整的仓库副本
- 大多数操作在本地完成
- 支持离线工作
- 无单点故障

优点:

- 速度快（本地操作）
- 支持离线工作
- 分支和合并高效
- 更好的容错性

缺点:

- 学习曲线较陡
- 仓库占用空间较大
- 权限管理复杂

TIP

现代软件开发中，分布式版本控制已成为主流，Git 占据了超过 90% 的市场份额。

1.2 Git 的发展历程

- 2005 年：Linux Torvalds 创建 Git，替代 BitKeeper
- 2008 年：GitHub 上线，推动 Git 普及
- 2010 年后：Git 成为行业标准
- 现在：几乎所有开源项目和大多数企业都使用 Git

2 Git 核心概念

2.1 四个工作区域

Git 有四个主要的工作区域，理解它们是掌握 Git 的关键：

工作区（Working Directory）

- 你正在编辑的文件所在的目录
- 包含项目的当前状态
- 可以随意修改文件

暂存区（Staging Area / Index）

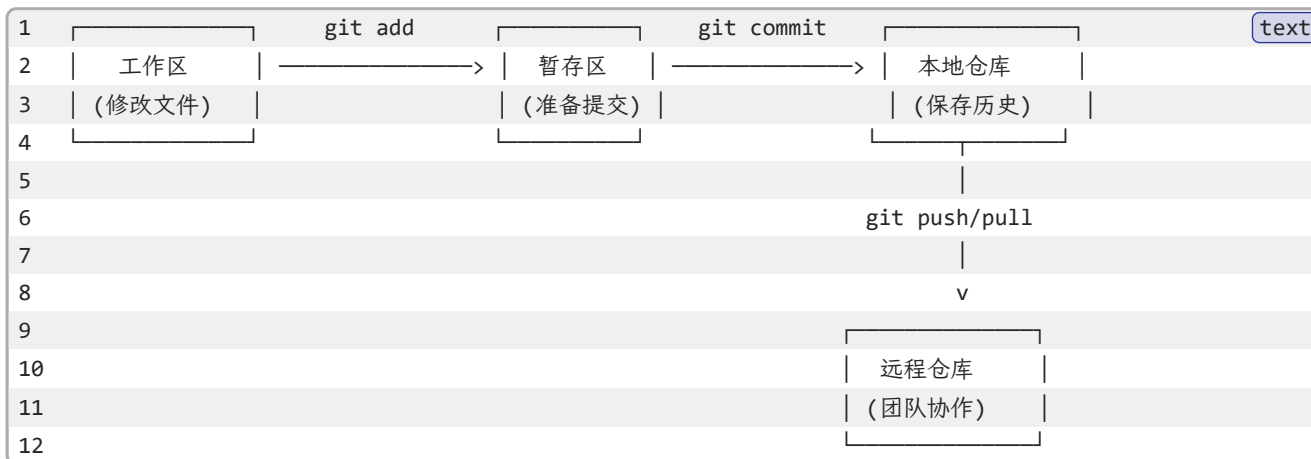
- 也称为“索引”
- 临时存储下次要提交的文件快照
- 通过 `git add` 将文件从工作区添加到暂存区
- 允许你精心准备每次提交的内容

本地仓库（Local Repository）

- 位于 `.git` 目录下
- 存储所有提交历史和元数据
- 是你的完整项目历史
- 通过 `git commit` 将暂存区内容提交到本地仓库

远程仓库 (Remote Repository)

- 存储在远程服务器上的仓库副本
- 如 GitHub、GitLab、Gitee 等平台
- 通过 `git push` 和 `git pull` 同步



CAUTION

新手常犯的错误是直接将工作区的修改 commit，而忽略了 add 步骤。记住：先 add，再 commit！

2.2 文件状态

Git 中的文件有三种状态：

- 已修改 (modified)：文件已被修改但尚未暂存
- 已暂存 (staged)：文件已标记为下次提交的版本
- 已提交 (committed)：文件已安全存储在本地数据库中

2.3 Git 对象类型

Git 底层使用四种对象类型来存储数据：

Blob (二进制大对象)

- 存储文件内容
- 不包含文件名或元数据
- 相同内容的文件只存储一次

Tree (树对象)

- 表示目录结构
- 包含文件名、权限和指向 blob 或其他 tree 的指针
- 类似于文件系统的目录

Commit (提交对象)

- 包含指向 tree 对象的指针
- 包含作者、提交者、时间戳和提交信息
- 包含指向父提交的指针 (形成历史记录)

Tag (标签对象)

- 给特定提交打上永久标签
- 通常用于标记发布版本
- 包含标签名、消息和指向 commit 的指针

TIP

可以使用 `git cat-file -p <hash>` 查看任何 Git 对象的详细内容，深入了解 Git 内部工作原理。

3 Git 基本操作

3.1 初始化仓库

git init

在当前目录创建新的 Git 仓库：

```
1 git init
```

 Bash

会在当前目录创建 `.git` 隐藏目录，包含所有 Git 元数据。

git clone

克隆远程仓库到本地：

```
1 # 克隆仓库
2 git clone https://github.com/user/repo.git
3
4 # 克隆到指定目录
5 git clone https://github.com/user/repo.git my-project
6
7 # 克隆特定分支
8 git clone -b develop https://github.com/user/repo.git
```

 Bash

NOTE

`git clone` 会自动设置 origin 远程仓库，并检出默认分支（通常是 main 或 master）。

3.2 查看状态与历史

git status

显示工作区和暂存区的状态：

```
1 git status
2
3 # 简洁输出
4 git status -s
```

 Bash

输出示例：

```
1 On branch main
```

```
2 Your branch is up to date with 'origin/main'.
3
4 Changes not staged for commit:
5   (use "git add <file>..." to update what will be committed)
6   (use "git restore <file>..." to discard changes in working directory)
7       modified:   README.md
8
9 Untracked files:
10   (use "git add <file>..." to include in what will be committed)
11       new_file.txt
```

git log

查看提交历史：

```
1 # 基本日志
2 git log
3
4 # 一行显示
5 git log --oneline
6
7 # 图形化显示分支
8 git log --graph --oneline --all
9
10 # 显示统计信息
11 git log --stat
12
13 # 显示补丁
14 git log -p
15
16 # 最近n次提交
17 git log -n 5
```

git diff

查看文件差异：

```
1 # 工作区与暂存区的差异
2 git diff
3
4 # 暂存区与最新提交的差异
5 git diff --staged
6
7 # 两个提交之间的差异
8 git diff commit1 commit2
9
10 # 特定文件的差异
11 git diff filename
```

3.3 添加与提交

git add

将文件添加到暂存区：

```
1 # 添加特定文件
2 git add file.txt
3
4 # 添加多个文件
5 git add file1.txt file2.txt
6
7 # 添加所有修改的文件
8 git add .
9
10 # 交互式添加
11 git add -i
12
13 # 添加部分修改 (patch模式)
14 git add -p
```

TIP

使用 `git add -p` 可以逐块审查修改，选择性地暂存部分修改，非常适合精细控制提交内容。

git commit

将暂存区的内容提交到本地仓库：

```
1 # 基本提交
2 git commit -m "feat: add user authentication"
3
4 # 使用编辑器编写详细提交信息
5 git commit
6
7 # 跳过暂存区，直接提交所有修改的文件
8 git commit -a -m "fix: correct typo"
9
10 # 修改最后一次提交
11 git commit --amend
```

提交信息规范 (Conventional Commits)：

```
1 <type>(<scope>): <subject>
2
3 <body>
4
5 <footer>
```

常用 type：

- `feat`：新功能
- `fix`：修复 bug
- `docs`：文档更新
- `style`：代码格式（不影响功能）
- `refactor`：重构

- `test`: 测试相关
- `chore`: 构建过程或辅助工具变动

CAUTION

避免使用模糊的提交信息如“update”、“fix bug”。好的提交信息应该清晰说明做了什么和为什么做。

3.4 撤销操作

git restore

撤销工作区的修改：

```
1 # 撤销单个文件的修改
2 git restore file.txt
3
4 # 撤销所有未暂存的修改
5 git restore .
```

 Bash

git reset

重置暂存区或回退提交：

```
1 # 软重置：保留工作区和暂存区
2 git reset --soft HEAD~1
3
4 # 混合重置（默认）：保留工作区，清空暂存区
5 git reset HEAD~1
6
7 # 硬重置：丢弃工作区和暂存区的修改
8 git reset --hard HEAD~1
9
10 # 重置特定文件
11 git reset HEAD file.txt
```

 Bash**CAUTION**

`git reset --hard` 会永久丢失未提交的修改，使用前务必确认！

git revert

创建新的提交来撤销之前的修改（安全的方式）：

```
1 # 撤销某次提交
2 git revert <commit-hash>
3
4 # 撤销多次提交
5 git revert HEAD~3..HEAD
```

 Bash**TIP**

对于已经 push 到远程的提交，使用 `git revert` 而非 `git reset`，避免破坏共享历史。

4 分支管理

分支是 Git 最强大的特性之一，允许你并行开发多个功能。

4.1 分支基本概念

Git 中的分支本质上是一个指向特定提交的轻量级指针。创建、切换和删除分支都非常快速。

```
1      A---B---C  main
2      \
3      D---E  feature
```

4.2 基本分支操作

查看分支

```
1 # 查看所有本地分支
2 git branch
3
4 # 查看所有分支（包括远程）
5 git branch -a
6
7 # 查看分支及其最后提交
8 git branch -v
9
10 # 查看已合并的分支
11 git branch --merged
12
13 # 查看未合并的分支
14 git branch --no-merged
```

创建分支

```
1 # 创建新分支
2 git branch feature-login
3
4 # 创建并切换到新分支
5 git checkout -b feature-login
6
7 # 基于特定提交创建分支
8 git branch feature-login abc1234
```

切换分支

```
1 # 切换到已有分支
2 git checkout feature-login
3
4 # 或使用新命令
5 git switch feature-login
6
```

```
7 # 切换回上一个分支
8 git checkout -
9 git switch -
```

TIP

Git 2.23+引入了 `git switch` 和 `git restore`，使命令语义更清晰。`git checkout` 仍然可用但功能过于复杂。

删除分支

```
1 # 删除已合并的分支
2 git branch -d feature-login
3
4 # 强制删除未合并的分支
5 git branch -D feature-login
6
7 # 删除远程分支
8 git push origin --delete feature-login
```

 Bash

4.3 合并分支

git merge

将一个分支的更改合并到当前分支：

```
1 # 切换到目标分支
2 git checkout main
3
4 # 合并feature分支
5 git merge feature-login
```

 Bash

合并策略：

- Fast-forward：如果目标分支是源分支的直接祖先，只是移动指针
- Three-way merge：创建一个新的 merge commit
- Squash merge：将所有提交压缩为一个

```
1 # 禁用fast-forward，总是创建merge commit
2 git merge --no-ff feature-login
3
4 # Squash合并
5 git merge --squash feature-login
6 git commit -m "feat: add login feature"
```

 Bash

解决冲突

当两个分支修改了同一文件的同一部分时，会产生冲突：

```
1 # 查看冲突文件
2 git status
3
4 # 手动编辑冲突文件，解决冲突后
5 git add resolved_file.txt
```

 Bash

```
6 git commit
```

冲突标记:

```
1 <<<<<< HEAD
2 你的修改
3 别人的修改
4 >>>>>> feature-branch
```

TIP

使用 `git mergetool` 可以调用可视化的合并工具（如 meld、kdiff3）来解决冲突。

4.4 Rebase（变基）

Rebase 可以将一个分支的提交“重新播放”到另一个分支上，保持线性历史。

```
1 # 将当前分支rebase到main
2 git checkout feature
3 git rebase main
```

Merge vs Rebase:

特性	Merge	Rebase
历史记录	保留完整历史，有 merge commit	线性历史，更清晰
真实性	反映真实的开发过程	重写历史
安全性	安全，不改变历史	可能危险，特别是已 push 的提交
适用场景	公共分支	私有分支

CAUTION

永远不要 rebase 已经 push 到远程的公共分支！这会改写历史，导致其他协作者的仓库出现问题。

I 交互式 Rebase

可以修改、合并、删除或重新排序提交：

```
1 git rebase -i HEAD~3
```

可用命令：

- `pick`: 保留提交
- `reword`: 保留提交，修改提交信息
- `edit`: 暂停，允许修改提交
- `squash`: 合并到前一个提交
- `fixup`: 类似 squash，但丢弃提交信息
- `drop`: 删除提交

TIP

交互式 rebase 是整理提交历史的强大工具，可以让你的提交历史更加清晰和专业。

5 远程协作

5.1 远程仓库管理

查看远程仓库

```
1 # 查看远程仓库
2 git remote -v
3
4 # 查看远程仓库详细信息
5 git remote show origin
```

 Bash

添加远程仓库

```
1 # 添加远程仓库
2 git remote add origin https://github.com/user/repo.git
3
4 # 添加多个远程仓库
5 git remote add upstream https://github.com/original/repo.git
```

 Bash

重命名和删除远程仓库

```
1 # 重命名
2 git remote rename origin github
3
4 # 删除
5 git remote remove github
```

 Bash

5.2 推送与拉取

git push

将本地提交推送到远程仓库：

```
1 # 推送到默认远程和分支
2 git push
3
4 # 推送到特定远程和分支
5 git push origin main
6
7 # 首次推送，设置上游分支
8 git push -u origin main
9
10 # 强制推送（谨慎使用）
11 git push --force
12
13 # 更安全的强制推送（只覆盖没有其他人推送的部分）
14 git push --force-with-lease
```

 Bash

CAUTION

避免使用 `--force`，优先使用 `--force-with-lease`，它会检查远程分支是否被其他人更新过。

git pull

从远程仓库拉取并合并更改：

```
1 # 拉取并合并
2 git pull
3
4 # 拉取特定分支
5 git pull origin main
6
7 # 拉取但不自动合并
8 git pull --no-commit
9
10 # 使用rebase代替merge
11 git pull --rebase
```

git fetch

仅下载远程更改，不自动合并：

```
1 # 获取所有远程分支的更新
2 git fetch
3
4 # 获取特定远程
5 git fetch origin
6
7 # 获取特定分支
8 git fetch origin main
```

TIP

推荐工作流程：先 `git fetch` 查看远程变化，再决定如何合并，而不是直接使用 `git pull`。

5.3 跟踪分支

```
1 # 设置上游分支
2 git branch --set-upstream-to=origin/main main
3
4 # 简写
5 git branch -u origin/main
6
7 # 查看跟踪关系
8 git branch -vv
```

6 高级操作

6.1 Stash（储藏）

临时保存工作进度，稍后恢复：

```
1 # 储藏当前修改
2 git stash
3
4 # 储藏并包含未跟踪的文件
5 git stash -u
6
7 # 带消息的储藏
8 git stash save "WIP: working on login feature"
9
10 # 查看所有储藏
11 git stash list
12
13 # 应用最近的储藏
14 git stash apply
15
16 # 应用并删除储藏
17 git stash pop
18
19 # 应用特定的储藏
20 git stash apply stash@{2}
21
22 # 删除储藏
23 git stash drop stash@{1}
24
25 # 清空所有储藏
26 git stash clear
```

NOTE

Stash 非常适合需要临时切换分支但又不想提交未完成工作的场景。

6.2 Cherry-pick

选择性地应用某个提交到当前分支：

```
1 # 应用单个提交
2 git cherry-pick abc1234
3
4 # 应用多个提交
5 git cherry-pick abc1234 def5678
6
7 # 应用提交范围
8 git cherry-pick abc1234..def5678
9
10 # 应用但不自动提交
11 git cherry-pick --no-commit abc1234
```

TIP

Cherry-pick 常用于将 bug 修复从开发分支应用到生产分支。

6.3 Reset 深入

```
1 # 软重置：只移动HEAD指针
2 git reset --soft HEAD~1
3
4 # 混合重置：移动HEAD并重置暂存区（默认）
5 git reset HEAD~1
6
7 # 硬重置：移动HEAD、重置暂存区和工作区
8 git reset --hard HEAD~1
9
10 # 重置到特定提交
11 git reset --hard abc1234
12
13 # 重置特定文件
14 git reset HEAD file.txt
```

6.4 Reflog（引用日志）

记录 HEAD 和分支引用的变化历史，是“后悔药”：

```
1 # 查看reflog
2 git reflog
3
4 # 查看特定分支的reflog
5 git reflog show main
6
7 # 恢复到之前的状态
8 git reset --hard HEAD@{3}
9
10 # 恢复丢失的提交
11 git reset --hard abc1234
```

CAUTION

Reflog 只在本地有效，默认保留 90 天。如果执行了 `git gc` 或超过了保留期限，可能无法恢复。

TIP

即使使用了 `git reset --hard` 或删除了分支，只要 reflog 还在，就有可能找回丢失的提交。

7 标签管理

标签用于标记重要的提交点，通常用于发布版本。

7.1 标签类型

轻量标签（Lightweight）

只是一个指向提交的指针：

```
1 git tag v1.0.0
```

 Bash

I 附注标签 (Annotated)

包含完整信息的标签对象：

```
1 git tag -a v1.0.0 -m "Release version 1.0.0"
```

 Bash

TIP

推荐使用附注标签，因为它包含更多信息（打标签的人、日期、消息等）。

7.2 标签操作

```
1 # 查看所有标签
2 git tag
3
4 # 查看匹配模式的标签
5 git tag -l "v1.*"
6
7 # 查看标签详情
8 git show v1.0.0
9
10 # 删除标签
11 git tag -d v1.0.0
12
13 # 推送标签到远程
14 git push origin v1.0.0
15
16 # 推送所有标签
17 git push origin --tags
18
19 # 删除远程标签
20 git push origin --delete v1.0.0
21
22 # 基于特定提交创建标签
23 git tag -a v1.0.0 abc1234 -m "Release version 1.0.0"
```

 Bash

8 Git 工作流

8.1 Git Flow

经典的 Git 分支模型，适合有固定发布周期的项目：

分支类型：

- `main/master`：生产环境代码
- `develop`：开发主分支
- `feature/*`：功能分支，从 develop 分出
- `release/*`：发布分支，从 develop 分出

- `hotfix/*`: 热修复分支, 从 main 分出

工作流程:

1. 从develop创建feature分支
2. 在feature分支上开发
3. 完成后合并回develop
4. 从develop创建release分支进行测试
5. release完成后合并到main和develop
6. 在main上打tag
7. 如有紧急bug, 从main创建hotfix分支
8. hotfix完成后合并到main和develop

NOTE

Git Flow 适合传统软件项目, 但对于持续部署的现代 Web 应用可能过于复杂。

8.2 GitHub Flow

更简单的分支模型, 适合持续部署:

核心原则:

- main 分支始终可部署
- 从 main 创建功能分支
- 频繁推送到远程
- 通过 Pull Request 讨论和审查
- 审查通过后合并到 main
- 立即部署

工作流程:

1. 从main创建分支
2. 提交更改
3. 打开Pull Request
4. 讨论和审查
5. 合并到main
6. 部署

TIP

GitHub Flow 简单灵活, 适合大多数现代软件开发项目, 特别是 SaaS 应用。

8.3 GitLab Flow

结合 Git Flow 和 GitHub Flow 的优点:

特点:

- 以 main 为主开发分支
- 可以有环境分支 (如 production、staging)
- 可以有版本分支 (如 v1.0、v2.0)
- 变更从上游向下游流动

适用场景:

- 需要同时维护多个版本
- 有多个部署环境
- 需要严格的发布管理

8.4 选择建议

工作流	复杂度	适用场景	学习成本
Git Flow	高	传统软件、固定发布周期	高
GitHub Flow	低	持续部署、Web 应用	低
GitLab Flow	中	多版本、多环境	中

TIP

小团队或新项目建议从 GitHub Flow 开始，随着项目复杂度增加再考虑更复杂的工作流。

9 .gitignore 配置

`.gitignore` 文件指定 Git 应该忽略哪些文件和目录。

9.1 基本语法

```
1 # 注释
2
3 # 忽略所有 .log 文件
4 *.log
5
6 # 忽略 node_modules 目录
7 node_modules/
8
9 # 忽略特定文件
10 config/local.json
11
12 # 例外规则（不忽略 important.log）
13 *.log
14 !important.log
15
16 # 忽略所有 txt 文件，除了 readme.txt
17 *.txt
18 !readme.txt
```

9.2 常用忽略规则

```
1 # 依赖目录
2 node_modules/
3 vendor/
4 __pycache__/
5
```

```
6 # 构建输出
7 dist/
8 build/
9 *.o
10 *.pyc
11
12 # 环境变量
13 .env
14 .env.local
15
16 # IDE配置
17 .vscode/
18 .idea/
19 *.swp
20 *.swo
21
22 # 操作系统文件
23 .DS_Store
24 Thumbs.db
25
26 # 日志文件
27 *.log
28 logs/
29
30 # 临时文件
31 tmp/
32 temp/
```

9.3 全局.gitignore

创建全局忽略文件，对所有仓库生效：

```
1 # 创建全局gitignore
2 git config --global core.excludesfile ~/.gitignore_global
3
4 # 编辑文件
5 vim ~/.gitignore_global
```

 Bash

TIP

将 IDE 配置、操作系统文件等个人偏好设置放在全局.gitignore 中，避免污染项目级别的配置。

9.4 .gitignore 最佳实践

- 在项目根目录放置 `.gitignore`
- 使用在线生成器（如 gitignore.io）生成模板
- 定期审查和更新
- 不要忽略必要的配置文件
- 团队成员应保持一致的忽略规则

10 Git 钩子 (Hooks)

Git 钩子是脚本，在特定事件发生时自动执行。

10.1 钩子类型

客户端钩子

- `pre-commit`: 提交前执行
- `prepare-commit-msg`: 准备提交消息时
- `commit-msg`: 验证提交消息
- `post-commit`: 提交后执行
- `pre-push`: 推送前执行

服务端钩子

- `pre-receive`: 接收推送前
- `update`: 更新引用前
- `post-receive`: 接收推送后

10.2 使用示例

pre-commit 钩子

在 `.git/hooks/pre-commit` 中:

```
1  #!/bin/bash
2
3  # 检查是否有调试代码
4  if grep -r "console.log" --include="*.js" .; then
5      echo "Error: Found console.log statements"
6      exit 1
7  fi
8
9  # 运行代码格式化
10 npm run lint
11
12 # 运行测试
13 npm test
```

```
1  # 使钩子可执行
2  chmod +x .git/hooks/pre-commit
```

NOTE

Git 钩子不会被 clone，需要在每个仓库中单独设置。可以使用工具如 Husky (Node.js) 来自动化这个过程。

commit-msg 钩子

验证提交消息格式:


```
1  #!/bin/bash
2
3  commit_msg=$(cat $1)
4
5  # 检查是否符合Conventional Commits规范
6  if ! echo "$commit_msg" | grep -qE "^(feat|fix|docs|style|refactor|test|chore)(\(.+\))?: .+";
   then
7      echo "Error: Commit message does not follow Conventional Commits format"
8      echo "Format: <type>(<scope>): <subject>"
9      exit 1
10 fi
```

10.3 Husky (推荐)

对于 Node.js 项目，使用 Husky 管理 Git 钩子：

```
1  # 安装Husky
2  npm install husky --save-dev
3  npx husky install
4
5  # 添加pre-commit钩子
6  npx husky add .husky/pre-commit "npm test"
7
8  # 添加commit-msg钩子
9  npx husky add .husky/commit-msg 'npx --no-install commitlint --edit "$1"'
```

TIP

使用 Husky + lint-staged 可以在提交前自动运行 linter 和测试，确保代码质量。

11 常见问题与解决方案

11.1 合并冲突解决

预防冲突

- 频繁 pull 远程更改
- 小而频繁的提交
- 清晰的沟通（谁在修改什么）
- 使用功能分支

解决冲突的步骤

```
1  # 1. 查看冲突文件
2  git status
3
4  # 2. 打开冲突文件，手动解决冲突
5  # 查找 <<<<<<<, =====, >>>>>>> 标记
6
7  # 3. 标记冲突已解决
```

```
8  git add resolved_file.txt
9
10 # 4. 完成合并
11  git commit
```

使用工具

```
1  # 使用内置合并工具
2  git mergetool
3
4  # 配置喜欢的合并工具
5  git config --global merge.tool meld
```

 Bash

11.2 修改提交历史

修改最后一次提交

```
1  # 修改提交消息
2  git commit --amend -m "New commit message"
3
4  # 添加遗漏的文件
5  git add forgotten_file.txt
6  git commit --amend --no-edit
```

 Bash

CAUTION

只修改还未 push 的提交！修改已 push 的提交会影响其他协作者。

修改多次提交

使用交互式 rebase：

```
1  git rebase -i HEAD~3
```

 Bash

然后根据需要选择 `reword`、`edit`、`squash` 等操作。

11.3 恢复丢失的提交

```
1  # 1. 查看reflog
2  git reflog
3
4  # 2. 找到丢失的提交的hash
5  git show abc1234
6
7  # 3. 恢复提交
8  git cherry-pick abc1234
9  # 或
10 git reset --hard abc1234
```

 Bash

11.4 清理仓库

```
1 # 删除未跟踪的文件
2 git clean -n # 预览
3 git clean -f # 执行
4
5 # 删除未跟踪的目录
6 git clean -fd
7
8 # 优化仓库
9 git gc
10
11 # 清理远程追踪分支
12 git remote prune origin
```

12 Git 性能优化

12.1 大仓库优化

浅克隆

```
1 # 只克隆最近的历史
2 git clone --depth 1 https://github.com/user/repo.git
3
4 # 克隆特定深度的历史
5 git clone --depth 50 https://github.com/user/repo.git
```

稀疏检出

```
1 # 只检出特定目录
2 git clone --sparse https://github.com/user/repo.git
3 cd repo
4 git sparse-checkout set src/docs
```

12.2 加速 Git 操作

启用 FSMonitor

```
1 git config core.fsmonitor true
```

使用 SSD

Git 操作大量涉及小文件读写，SSD 能显著提升性能。

增加内存缓存

```
1 git config --global core.preloadIndex true
2 git config --global core.unpackLimit 10000
```

12.3 减小仓库大小

```
1 # 清理不必要的对象
2 git gc --prune=now --aggressive
3
4 # 移除大文件历史（使用git-filter-repo）
5 pip install git-filter-repo
6 git filter-repo --path large_file.bin --invert-paths
```

CAUTION

修改历史后需要强制推送，并确保团队成员知道如何处理。

12.4 分包（Submodules）

将大型依赖作为子模块：

```
1 # 添加子模块
2 git submodule add https://github.com/user/lib.git libs/mylib
3
4 # 初始化子模块
5 git submodule init
6 git submodule update
7
8 # 克隆包含子模块的仓库
9 git clone --recursive https://github.com/user/repo.git
```

TIP

对于现代项目，考虑使用包管理器（npm、pip、maven 等）代替 submodules，更简单易用。

13 Git 最佳实践总结

13.1 提交规范

- 小而频繁的提交
- 清晰的提交信息
- 一个提交只做一件事
- 遵循 Conventional Commits 规范

13.2 分支策略

- 使用有意义的分支名
- 及时删除已合并的分支
- 定期 rebase 以保持历史清晰
- 不要 rebase 公共分支

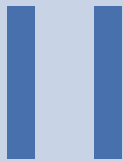
13.3 协作礼仪

- 经常 push 和 pull
- 代码审查后再合并
- 尊重他人的工作
- 清晰的 PR 描述

13.4 安全措施

- 使用 SSH 密钥而非 HTTPS 密码
- 启用双因素认证
- 定期备份重要仓库
- 谨慎使用 force push

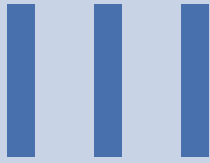
本章完



数据库系统

Chapter 2

数据库基础



并发编程与分布式系统

3.	并发编程基础·····	33
4.	分布式系统理论·····	34

Chapter 3

并发编程基础

Chapter 4

分布式系统理论

分布式系统是由多台计算机通过网络协同工作，对外表现为一个统一系统的软件架构。它在现代互联网应用中无处不在，从搜索引擎到社交媒体，从电子商务到云计算平台。

NOTE

分布式系统的核心挑战在于不确定性：网络延迟、节点故障、时钟不同步等问题使得分布式系统比单机系统复杂得多。

1 分布式系统概述与挑战

1.1 什么是分布式系统

分布式系统是一组独立的计算机，对用户来说就像是一个单一的系统。

关键特征：

- 并发性：多个组件同时执行
- 缺乏全局时钟：没有统一的时序参考
- 独立故障：组件可能独立失败
- 消息传递：通过消息进行通信

1.2 为什么需要分布式系统

可扩展性 (Scalability)

- 垂直扩展：增加单机资源（CPU、内存）
- 水平扩展：增加机器数量
- 分布式系统支持水平扩展，理论上可以无限扩展

可用性 (Availability)

- 单点故障会导致整个系统不可用
- 分布式系统通过冗余提高可用性
- 即使部分节点故障，系统仍可运行

性能 (Performance)

- 地理分布减少延迟
- 并行处理提高吞吐量
- 负载均衡优化资源利用

容错性 (Fault Tolerance)

- 冗余设计避免单点故障
- 自动故障检测和恢复
- 数据复制保证可靠性

1.3 分布式系统的挑战

网络问题

- 网络延迟: 消息传输需要时间
- 网络分区: 网络可能分裂成多个部分
- 带宽限制: 数据传输速率有限
- 不可靠性: 消息可能丢失、重复或乱序

CAUTION

网络是不可靠的! 永远不要假设消息一定会到达, 也不要假设只会到达一次。

时钟同步

- 不同节点的时钟可能不同步
- 无法确定事件的绝对顺序
- 需要使用逻辑时钟或向量时钟

一致性问题

- 如何在多个副本之间保持一致
- CAP 定理的限制
- 一致性 vs 可用性的权衡

部分故障

- 某些节点可能故障, 其他节点正常
- 难以区分是节点故障还是网络延迟
- 需要复杂的故障检测机制

并发控制

- 多个客户端同时访问共享资源
- 需要分布式锁或事务
- 死锁检测和处理更复杂

TIP

Leslie Lamport 的名言: “分布式系统是这样一個系统, 其中你不知道的计算机发生故障, 导致你无法完成自己的工作。”

2 CAP 定理详解

CAP 定理是分布式系统理论的基石，由 Eric Brewer 于 2000 年提出，并在 2002 年由 Seth Gilbert 和 Nancy Lynch 证明。

2.1 CAP 三要素

一致性 (Consistency)

定义：所有节点在同一时刻看到的数据是相同的。

特点：

- 读操作总是返回最新的写操作结果
- 强一致性保证
- 类似于单机数据库的 ACID 中的 C

示例：

- 1 客户端A写入 $X = 1$
- 2 客户端B立即读取 X ，必须得到 1

可用性 (Availability)

定义：每个请求都能在合理时间内收到响应（成功或失败）。

特点：

- 系统始终可用，不会拒绝服务
- 不保证返回最新数据
- 关注响应时间

示例：

- 1 客户端发送请求
- 2 系统必须在超时时间内返回响应
- 3 即使返回的是旧数据，也算可用

分区容错性 (Partition Tolerance)

定义：系统在遇到网络分区时仍能继续运行。

特点：

- 网络分区是不可避免的
- 系统必须能够容忍分区
- 实际分布式系统中 P 是必须的

示例：

- 1 网络分成两部分：{A, B} 和 {C, D}
- 2 两部分之间无法通信
- 3 系统仍需在各分区内正常工作

NOTE

在现实世界中，网络分区是必然发生的（路由器故障、光缆被挖断等），因此 P 是分布式系统必须具备的特性。

2.2 CAP 定理的核心结论

一个分布式系统最多只能同时满足 CAP 中的两个特性。

1	C (一致性)	text
2	/ \	
3	/ \	
4	/ \	
5	A-----P	
6	(可用性) (分区容错性)	
7		
8	只能选择两条边，不能三者兼得	

2.3 三种组合

CP 系统（一致性 + 分区容错性）

特点：

- 保证数据一致性
- 发生分区时，不可用的分区会拒绝服务
- 牺牲可用性来保证一致性

典型应用：

- 银行系统
- 分布式数据库（HBase、MongoDB 默认配置）
- ZooKeeper
- etcd

场景示例：

1	网络分区发生
2	主节点在分区A，从节点在分区B
3	分区B的节点拒绝读写请求（保证一致性）
4	只有分区A可以正常服务

AP 系统（可用性 + 分区容错性）

特点：

- 保证系统始终可用
- 允许数据暂时不一致
- 最终会达到一致状态

典型应用：

- DNS 系统
- Cassandra
- DynamoDB
- CDN

场景示例：

1	网络分区发生
---	--------

- 2 两个分区都可以接受写请求
- 3 可能导致数据冲突
- 4 分区恢复后通过某种策略解决冲突

CA 系统（一致性 + 可用性）

特点：

- 理论上存在，但实际中不存在
- 因为没有分区容错性
- 只能在单机或非分布式系统中实现

CAUTION

CA 系统在实际的分布式系统中是不存在的，因为网络分区是不可避免的。任何声称是 CA 的“分布式系统”实际上都不是真正的分布式系统。

2.4 CAP 的实际意义

CAP 定理告诉我们：

1. 没有完美的系统：必须做出权衡
2. 根据业务需求选择：不同场景需要不同的权衡
3. 动态调整：可以在不同情况下选择不同的策略
4. 理解代价：知道选择某个特性意味着放弃什么

TIP

大多数现代分布式系统都是 AP 或 CP，具体选择取决于业务需求。例如，社交网络倾向于 AP（可以接受短暂的不一致），而金融系统倾向于 CP（必须保证一致性）。

3 CAP 权衡与实际应用

3.1 如何选择合适的 CAP 组合

考虑因素

业务需求

- 金融交易：需要 CP（一致性优先）
- 社交动态：可以 AP（可用性优先）
- 电商库存：需要 CP（避免超卖）
- 用户资料：可以 AP（最终一致即可）

用户体验

- 实时性要求高：可能需要 CP
- 可以接受延迟：可以选择 AP
- 离线可用：需要 AP

技术约束

- 数据量大小
- 读写比例
- 地理分布
- 团队技术能力

3.2 实际应用案例

案例 1：电商平台

需求分析：

- 商品浏览：AP（高可用，可以接受短暂不一致）
- 购物车：AP（用户可以稍后同步）
- 订单创建：CP（必须保证一致性，避免重复订单）
- 库存扣减：CP（避免超卖）
- 支付处理：CP（绝对不能出错）

解决方案：

- 1 使用混合架构：
- 2 - 商品缓存：Redis（AP）
- 3 - 订单系统：关系型数据库 + 分布式事务（CP）
- 4 - 库存系统：Redis + 数据库最终一致（AP -> CP）
- 5 - 支付系统：强一致性数据库（CP）

案例 2：社交网络

需求分析：

- 发布动态：AP（快速响应，最终一致）
- 点赞评论：AP（可以接受短暂不一致）
- 好友关系：CP（需要一致性）
- 消息推送：AP（允许丢失少量消息）

解决方案：

- 1 - 动态存储：Cassandra（AP）
- 2 - 关系数据：MySQL主从（最终一致）
- 3 - 实时消息：Kafka + WebSocket（AP）

案例 3：协作编辑工具

需求分析：

- 文档编辑：需要高可用和低延迟
- 多人协作：需要解决冲突
- 历史版本：需要一致性

解决方案：

- 1 - 使用CRDT（无冲突复制数据类型）
- 2 - 操作转换（OT）算法
- 3 - 最终一致性模型
- 4 - 定期快照保证长期一致性

NOTE

Google Docs、Figma 等协作工具都使用了类似的技术来实现多人实时协作。

3.3 超越 CAP: PACELC 定理

CAP 定理只考虑了网络分区的情况，PACELC 定理进行了扩展：

If Partition (P), then trade-off between Availability (A) and Consistency (C); Else (E), trade-off between Latency (L) and Consistency (C).

含义：

- 如果发生分区 (P)，在可用性 (A) 和一致性 (C) 之间权衡
- 否则 (E)，在延迟 (L) 和一致性 (C) 之间权衡

意义：

- 即使没有分区，也需要在延迟和一致性之间权衡
- 更全面地描述了分布式系统的权衡

TIP

PACELC 定理提醒我们，即使在正常情况下，也需要考虑延迟和一致性的平衡，而不仅仅是分区时的选择。

4 BASE 理论

BASE 理论是对 CAP 中 AP 系统的进一步阐述，由 eBay 的 Dan Pritchett 提出。

4.1 BASE 三要素

基本可用 (Basically Available)

定义：分布式系统在出现故障时，允许损失部分可用性，但保证核心功能可用。

特点：

- 不是 100% 可用，而是“基本”可用
- 允许响应时间增加
- 允许功能降级

示例：

- 1 双11购物节：
- 2 - 核心功能（下单、支付）保持可用
- 3 - 非核心功能（推荐、评价）可能降级
- 4 - 响应时间可能变慢，但不会完全不可用

软状态 (Soft State)

定义：允许系统中的数据存在中间状态，这些状态不影响系统整体可用性。

特点：

- 数据可以有临时不一致
- 状态会随着时间推移而变化

- 不需要立即达到一致

示例：

- 1 用户修改头像：
- 2 1. 上传到新服务器A
- 3 2. CDN节点B仍然是旧头像
- 4 3. 几分钟后，CDN同步完成
- 5 4. 期间存在“软状态”

最终一致性（Eventual Consistency）

定义：系统中的数据副本在经过一段时间后，最终会达到一致状态。

特点：

- 不保证即时一致
- 保证最终会一致
- 一致性窗口期内的不一致是可接受的

示例：

- 1 DNS更新：
- 2 - 修改DNS记录
- 3 - 全球DNS服务器需要时间传播
- 4 - 可能在几分钟到几小时内不一致
- 5 - 但最终所有服务器都会更新

NOTE

BASE 理论的核心思想是：通过牺牲强一致性来获得高可用性和高性能，适用于大多数互联网应用场景。

4.2 最终一致性的实现方式

读写一致性级别

读己之所写（Read-your-writes）

用户总能读取到自己写入的最新数据。

实现：

- 将用户的读写请求路由到同一节点
- 或使用 session 粘滞

会话一致性（Session Consistency）

在同一会话中，保证读己之所写。

实现：

- 基于 session 的路由
- 客户端缓存

单调读一致性（Monotonic Read Consistency）

如果用户已经读取过某个数据的值，后续的读取不会返回更早的值。

实现:

- 版本向量
- 因果依赖跟踪

单调写一致性 (Monotonic Write Consistency)

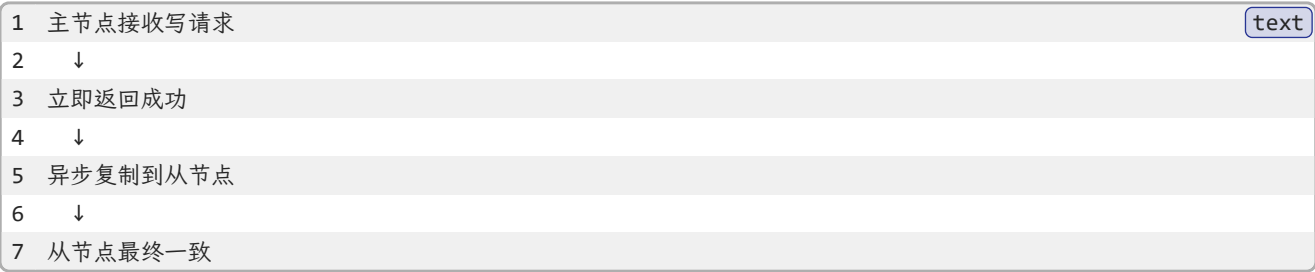
保证写操作的顺序执行。

实现:

- 队列化写操作
- 序列号

常见实现模式

异步复制



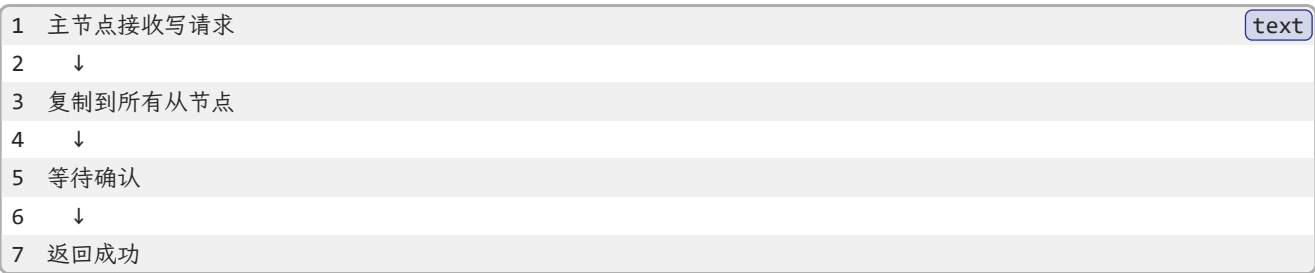
优点:

- 写性能好
- 延迟低

缺点:

- 可能丢失数据 (主节点故障)
- 读取可能不一致

同步复制



优点:

- 数据安全性高
- 一致性强

缺点:

- 写性能差
- 延迟高

半同步复制

折中方案:

- 至少复制到一个从节点
- 然后返回成功
- 平衡性能和安全性

TIP

MySQL 的主从复制默认是异步的，可以配置为半同步以提高数据安全性。

5 一致性模型

一致性模型定义了分布式系统中数据访问的行为规范。

5.1 强一致性模型

线性一致性 (Linearizability)

定义：所有操作看起来是原子性地在一个时间点执行，且符合实时顺序。

特点：

- 最强的一致性保证
- 等价于单线程执行
- 实现成本高

示例：

```
1 时间线:  $t_1 < t_2 < t_3$ 
2
3 t1: 客户端A写入  $X = 1$ 
4 t2: 客户端B读取  $X$ , 必须得到 1
5 t3: 客户端C读取  $X$ , 必须得到 1
```

应用：

- 分布式锁
- 领导者选举
- ZooKeeper、etcd

顺序一致性 (Sequential Consistency)

定义：所有进程看到的操作顺序相同，且符合程序顺序。

特点：

- 比线性一致性弱
- 不要求实时性
- 更容易实现

示例：

```
1 进程P1:  $W(x)1, W(y)1$ 
2 进程P2:  $R(y)1, R(x)1$ 
3
4 所有进程看到的顺序必须是:  $W(x)1, W(y)1, R(y)1, R(x)1$ 
```

5 但不能保证这个顺序与实际时间对应

5.2 弱一致性模型

I 因果一致性 (Causal Consistency)

定义：有因果关系的操作必须按顺序执行，无关操作可以任意顺序。

特点：

- 比顺序一致性弱
- 保留因果关系
- 性能较好

示例：

```
1 因果关系：
2 A发帖 -> B评论 -> C回复
3
4 这三个操作必须按顺序可见
5
6 但A发帖和D发帖（无关）可以任意顺序
```

实现：

- 向量时钟
- 依赖跟踪

I 最终一致性 (Eventual Consistency)

定义：如果没有新的更新，最终所有副本会达到一致状态。

特点：

- 最弱的一致性保证
- 最好的性能和可用性
- 可能存在冲突

应用：

- DNS
- CDN
- NoSQL 数据库 (Cassandra、DynamoDB)

5.3 一致性模型对比

模型	强度	性能	实现难度	典型应用
线性一致性	最强	最低	最难	分布式锁
顺序一致性	强	低	难	共享内存
因果一致性	中	中	中	协作编辑
最终一致性	弱	最高	最简单	DNS、CDN

TIP

选择一致性模型时，需要根据业务需求权衡。不是越强越好，而是要找到最适合的平衡点。

6 分布式事务

分布式事务确保跨多个节点的操作要么全部成功，要么全部失败，保持数据一致性。

6.1 ACID vs BASE

特性	ACID	BASE
一致性	强一致性	最终一致性
可用性	较低	较高
性能	较低	较高
适用场景	金融、传统企业	互联网、大数据
典型系统	关系型数据库	NoSQL、分布式系统

6.2 两阶段提交（2PC）

2PC 是最经典的分布式事务协议。

1 工作原理

参与者角色：

- 协调者（Coordinator）：负责协调事务
- 参与者（Participant）：执行具体操作

两个阶段：

第一阶段：准备阶段（Prepare Phase）

1	1. 协调者向所有参与者发送PREPARE请求
2	2. 参与者执行事务操作，但不提交
3	3. 参与者写入undo log和redo log
4	4. 参与者返回YES或NO
5	- YES: 准备就绪，可以提交
6	- NO: 准备失败，需要回滚

第二阶段：提交阶段（Commit Phase）

1	情况1: 所有参与者都返回YES
2	1. 协调者发送COMMIT请求
3	2. 参与者正式提交事务
4	3. 释放资源
5	4. 返回ACK
6	
7	情况2: 有参与者返回NO或超时
8	1. 协调者发送ROLLBACK请求
9	2. 参与者回滚事务
10	3. 释放资源
11	4. 返回ACK

1	协调者	参与者A	参与者B	text
2				
3	--- PREPARE ----->			
4	--- PREPARE ----->			
5				
6	<-- YES -----			
7	<-- YES -----			
8				
9	--- COMMIT ----->			
10	--- COMMIT ----->			
11				
12	<-- ACK -----			
13	<-- ACK -----			

优点

- 保证了强一致性
- 原理简单，易于理解
- 广泛应用

缺点

- 同步阻塞：所有参与者在等待期间都持有资源
- 单点故障：协调者故障会导致整个事务不确定
- 数据不一致：网络异常可能导致部分提交
- 性能问题：两次网络往返，延迟高

CAUTION

2PC 在生产环境中很少直接使用，因为它的性能问题和可靠性问题。通常作为更高级协议的基础。

6.3 三阶段提交（3PC）

3PC 在 2PC 的基础上增加了超时机制和一个预提交阶段。

三个阶段

CanCommit 阶段

协调者询问参与者是否可以执行事务。

PreCommit 阶段

参与者执行事务但不提交，进入准备状态。

DoCommit 阶段

正式提交或回滚。

改进点

- 增加了超时机制

- 减少了阻塞时间
- 降低了单点故障的影响

仍然存在的问题

- 网络分区时仍可能不一致
- 复杂度更高
- 实际应用较少

NOTE

3PC 理论上比 2PC 更好，但由于复杂度和实际问题，在实际系统中使用不多。

6.4 TCC (Try-Confirm-Cancel)

TCC 是一种应用层的补偿事务协议。

三个操作

Try (尝试)

- 检查业务活动是否可以执行
- 预留业务资源
- 不真正执行业务

Confirm (确认)

- 真正执行业务
- 不使用任何业务检查
- 幂等操作

Cancel (取消)

- 释放 Try 阶段预留的资源
- 补偿操作
- 幂等操作

示例：转账

```
1  # Try阶段
2  def try_transfer(from_account, to_account, amount):
3      # 检查余额
4      if from_account.balance < amount:
5          return False
6      # 冻结资金
7      from_account.frozen_balance += amount
8      return True
9
10 # Confirm阶段
11 def confirm_transfer(from_account, to_account, amount):
12     # 真正转账
13     from_account.balance -= amount
```

 Python

```
14     from_account.frozen_balance -= amount
15     to_account.balance += amount
16
17 # Cancel阶段
18 def cancel_transfer(from_account, amount):
19     # 解冻资金
20     from_account.frozen_balance -= amount
```

优点:

- 应用层控制, 灵活
- 避免了长时间锁定资源
- 适合业务流程复杂的场景

缺点:

- 需要实现三个接口, 开发成本高
- 需要考虑幂等性
- 空回滚、悬挂等问题需要处理

TIP

TCC 适合对一致性要求高、业务流程复杂的场景, 如金融系统、电商订单等。

6.5 Saga 模式

Saga 将长事务拆分为多个本地短事务, 每个事务都有对应的补偿操作。

两种编排方式

choreography (编舞式)

每个事务完成后发布事件, 触发下一个事务。

```
1  订单服务 --创建订单--> 事件总线
2
3          ↓
4          库存服务 <--订阅事件--
5          --扣减库存--> 事件总线
6
7          ↓
8          支付服务 <--订阅事件--
9          --发起支付-->
```

优点:

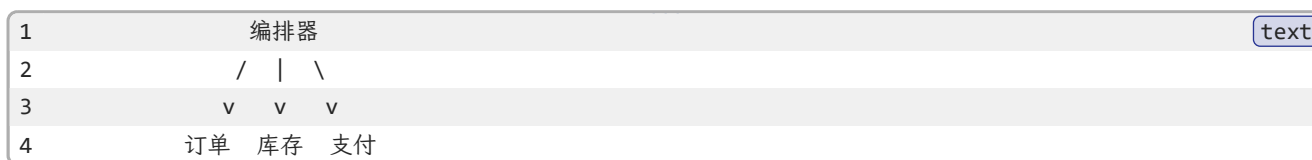
- 去中心化
- 松耦合

缺点:

- 流程难以追踪
- 循环依赖风险

Orchestration (编排式)

由一个编排器 (Orchestrator) 控制整个流程。



优点:

- 集中控制
- 易于监控和管理

缺点:

- 编排器可能成为瓶颈
- 耦合度较高

示例：电商订单

```
1 class OrderSaga:
2     def execute(self, order):
3         try:
4             # 步骤1: 创建订单
5             self.create_order(order)
6
7             # 步骤2: 扣减库存
8             self.deduct_inventory(order.items)
9
10            # 步骤3: 发起支付
11            self.initiate_payment(order)
12
13            # 步骤4: 更新订单状态
14            self.update_order_status(order, 'PAID')
15
16        except Exception as e:
17            # 补偿操作 (反向执行)
18            self.compensate(order)
19
20    def compensate(self, order):
21        # 退款
22        self.refund_payment(order)
23
24        # 恢复库存
25        self.restore_inventory(order.items)
26
27        # 取消订单
28        self.cancel_order(order)
```

优点:

- 适合长事务
- 每个步骤都是本地事务
- 灵活性高

缺点:

- 可能出现中间状态
- 补偿逻辑复杂

- 不保证隔离性

NOTE

Saga 模式在微服务架构中非常流行，特别适合业务流程长的场景。

6.6 分布式事务选型建议

协议	一致性	性能	复杂度	适用场景
2PC	强	低	中	传统企业系统
3PC	强	中	高	较少使用
TCC	最终	高	高	金融、电商
Saga	最终	高	中	微服务、长事务
本地消息表	最终	高	低	异步解耦
最大努力通知	最终	高	低	通知类业务

TIP

现代分布式系统更多使用最终一致性模型（TCC、Saga），而非强一致性模型（2PC、3PC）。

7 分布式锁实现

分布式锁用于在分布式环境中协调对共享资源的访问。

7.1 分布式锁的要求

- 互斥性：同一时刻只有一个客户端能持有锁
- 安全性：只有加锁的客户端才能解锁
- 死锁避免：即使客户端崩溃，锁也能释放
- 容错性：部分节点故障不影响锁的可用性

7.2 基于 Redis 的分布式锁

Redlock 算法

Redis 作者 antirez 提出的分布式锁算法。

实现步骤：

```
1 import redis
2 import time
3 import uuid
4
5 class Redlock:
6     def __init__(self, redis_nodes):
7         self.nodes = [redis.Redis(node) for node in redis_nodes]
8         self.quorum = len(redis_nodes) // 2 + 1
9         self.validity_time = 10000 # 锁有效期（毫秒）
10
```

 Python

```
11 def acquire_lock(self, resource, ttl):
12     token = str(uuid.uuid4())
13     start_time = int(time.time() * 1000)
14
15     # 尝试在大多数节点上加锁
16     locked_nodes = 0
17     for node in self.nodes:
18         if node.set(resource, token, nx=True, px=ttl):
19             locked_nodes += 1
20
21     # 计算有效时间
22     elapsed = int(time.time() * 1000) - start_time
23     validity = ttl - elapsed
24
25     # 判断是否成功
26     if locked_nodes >= self.quorum and validity > 0:
27         return {'token': token, 'validity': validity}
28     else:
29         # 失败则释放所有已加的锁
30         self.release_lock(resource, token)
31         return None
32
33 def release_lock(self, resource, token):
34     # 使用Lua脚本保证原子性
35     lua_script = """
36     if redis.call("get", KEYS[1]) == ARGV[1] then
37         return redis.call("del", KEYS[1])
38     else
39         return 0
40     end
41     """
42     for node in self.nodes:
43         node.eval(lua_script, 1, resource, token)
```

优点:

- 性能高
- 实现相对简单

缺点:

- 依赖时钟同步
- 网络延迟可能导致问题
- 需要多个 Redis 实例

CAUTION

Martin Kleppmann 曾对 Redlock 提出质疑, 认为在极端情况下可能不安全。实际使用时需要仔细评估风险。

Redisson 实现

Java 中最常用的 Redis 分布式锁实现:

```
1 RLock lock = redissonClient.getLock("myLock");
```



```
2  try {
3      // 尝试加锁，最多等待10秒，锁定30秒后自动释放
4      boolean res = lock.tryLock(10, 30, TimeUnit.SECONDS);
5      if (res) {
6          // 执行业务逻辑
7      }
8  } finally {
9      lock.unlock();
10 }
```

特性：

- 看门狗机制（自动续期）
- 可重入
- 公平锁
- 读写锁

7.3 基于 ZooKeeper 的分布式锁

实现原理

利用 ZooKeeper 的临时顺序节点特性。

步骤：

1. 创建临时顺序节点 /locks/lock-0000000001
2. 获取 /locks 下所有子节点
3. 判断自己是否是最小的节点
 - 是：获得锁
 - 否：监听前一个节点
4. 前一个节点删除时，重新检查
5. 释放锁：删除自己的节点

```
1  from kazoo.client import KazooClient
2
3  class ZkDistributedLock:
4      def __init__(self, zk_hosts, lock_path):
5          self.zk = KazooClient(hosts=zk_hosts)
6          self.zk.start()
7          self.lock_path = lock_path
8          self.lock_node = None
9
10     def acquire(self, timeout=10):
11         # 创建临时顺序节点
12         self.lock_node = self.zk.create(
13             self.lock_path + "/lock-",
14             ephemeral=True,
15             sequence=True
16         )
17
18         # 获取所有子节点并排序
19         children = sorted(self.zk.get_children(self.lock_path))
20         my_node = self.lock_node.split('/')[-1]
21         my_index = children.index(my_node)
```

 Python

```
22
23     if my_index == 0:
24         # 我是第一个，获得锁
25         return True
26     else:
27         # 监听前一个节点
28         prev_node = children[my_index - 1]
29         event = self.zk.DataWatch(
30             self.lock_path + "/" + prev_node
31         )
32         # 等待前一个节点删除
33         event.wait(timeout=timeout)
34         return True
35
36     def release(self):
37         if self.lock_node:
38             self.zk.delete(self.lock_node)
39             self.lock_node = None
```

优点：

- 可靠性高
- 不会出现脑裂
- ZooKeeper 本身提供一致性保证

缺点：

- 性能相对较低
- 频繁创建删除节点
- 羊群效应（大量客户端同时监听）

TIP

Curator 框架提供了完善的 ZooKeeper 分布式锁实现，推荐使用。

7.4 基于 etcd 的分布式锁

etcd 提供了原生的租约（Lease）和锁机制。

```
1  import etcd3
2
3  client = etcd3.client()
4
5  # 创建锁
6  lock = client.lock('my-lock')
7
8  # 获取锁
9  if lock.acquire(timeout=10):
10     try:
11         # 执行业务逻辑
12         pass
13     finally:
14         # 释放锁
15         lock.release()
```

 Python

优点:

- API 简洁
- 基于 Raft，一致性强
- 适合云原生环境

7.5 分布式锁对比

实现方式	性能	可靠性	复杂度	适用场景
Redis Redlock	高	中	中	高性能要求
ZooKeeper	中	高	中	强一致性要求
etcd	中	高	低	K8s、云原生
数据库	低	高	低	简单场景

NOTE

选择分布式锁实现时，需要权衡性能、可靠性和复杂度。对于关键业务，建议使用 ZooKeeper 或 etcd；对于高性能场景，可以考虑 Redis。

8 Leader 选举算法

Leader 选举是分布式系统中的常见问题，用于选出一个协调者。

8.1 为什么需要 Leader 选举

- 协调任务：需要一个节点来协调分布式操作
- 避免冲突：防止多个节点同时执行相同操作
- 简化设计：主从架构比多主架构更简单
- 数据一致性：通过单一写入点保证一致性

8.2 Bully 算法

最简单的 Leader 选举算法。

算法流程

- 1 1. 当发现Leader失效时，发起选举
- 2 2. 向所有ID比自己大的节点发送ELECTION消息
- 3 3. 如果没有响应，自己成为Leader
- 4 4. 如果有响应，等待对方的COORDINATOR消息
- 5 5. 收到COORDINATOR消息，承认对方为Leader

优点:

- 简单易懂
- 保证选出 ID 最大的节点

缺点:

- 消息复杂度高 $O(n^2)$

- 网络不稳定时可能频繁选举
- ID 需要全局唯一且有序

8.3 Ring 算法

节点组织成环状结构。

算法流程

1. 检测到Leader失效
2. 向环中下一个节点发送ELECTION消息
3. 消息中包含自己的ID
4. 每个节点比较ID，保留最大的
5. 消息回到发起者时，最大ID成为Leader

优点：

- 消息复杂度 $O(n)$
- 比 Bully 算法高效

缺点：

- 环维护复杂
- 单点故障影响大

8.4 Raft 选举

Raft 算法中的 Leader 选举机制。

选举过程

1. 节点启动时为Follower
2. 如果在选举超时时间内没收到Leader心跳
3. 转为Candidate，增加term
4. 向所有节点请求投票
5. 如果获得多数票，成为Leader
6. 开始发送心跳

特点：

- 基于多数派
- 随机超时避免选票分裂
- term 机制防止脑裂

TIP

Raft 是目前最流行的共识算法之一，etcd、Consul 等都使用 Raft。

8.5 ZooKeeper 的 Leader 选举

基于 ZAB 协议的选举机制。

I 选举过程

1. 每个服务器提出自己的ZXID和SID
2. 比较ZXID, 较大的优先
3. ZXID相同, 比较SID, 较大的优先
4. 统计投票, 超过半数成为Leader
5. 进入同步阶段

特点:

- 基于 ZXID (事务 ID)
- 保证数据最新的节点成为 Leader
- 快速故障转移

9 拜占庭将军问题

拜占庭将军问题是分布式系统中的一个经典问题, 由 Leslie Lamport 等人于 1982 年提出。

9.1 问题描述

场景:

- N 个将军围攻一座城市
- 将军们需要通过信使传递消息来协调行动
- 有些将军可能是叛徒, 会发送错误消息
- 忠诚的将军需要达成一致的行动方案

核心问题:

在存在恶意节点的情况下, 如何让诚实节点达成一致?

9.2 形式化定义

拜占庭故障:

- 节点可能以任意方式故障
- 可能发送错误消息
- 可能故意破坏系统
- 比崩溃故障更难处理

要求:

1. 所有诚实节点达成一致的决策
2. 如果所有诚实节点初始值相同, 则决策为该值

9.3 解决方案

I PBFT (实用拜占庭容错)

Miguel Castro 和 Barbara Liskov 于 1999 年提出。

前提条件:

- 总节点数 $n \geq 3f + 1$ (f 为故障节点数)
- 异步网络环境

- 节点身份已知

三个阶段：

Pre-prepare 阶段

主节点接收请求，广播给所有备份节点。

Prepare 阶段

备份节点验证消息，广播 prepare 消息。

Commit 阶段

节点收集到足够多的 prepare 消息后，广播 commit 消息。

1	客户端	主节点	备份节点	text
2				
3		--REQUEST--->		
4			--PRE-PREPARE->	
5			<---PREPARE----	
6			--PREPARE----->	
7			<---COMMIT-----	
8			--COMMIT----->	
9		<---REPLY-----		

优点：

- 解决了拜占庭问题
- 性能相对较好

缺点：

- 消息复杂度高 $O(n^2)$
- 只能容忍 $< n/3$ 的故障节点
- 扩展性差

NOTE

PBFT 主要用于联盟链等小规模区块链系统，不适合大规模公有链。

PoW（工作量证明）

比特币使用的共识机制。

原理：

- 通过计算哈希 puzzle 来达成共识
- 需要消耗大量计算资源
- 最长链原则

优点：

- 完全去中心化
- 安全性高
- 无需许可

缺点:

- 能耗巨大
- 吞吐量低
- 确认时间长

■ PoS (权益证明)

以太坊 2.0 采用的机制。

原理:

- 根据持币数量和时长选择验证者
- 验证者质押代币
- 作恶会被罚没

优点:

- 能耗低
- 吞吐量较高
- 更环保

缺点:

- 可能存在富者愈富
- Nothing at Stake 问题

9.4 拜占庭容错的应用

■ 区块链

- 比特币: PoW
- 以太坊: PoS (2.0)
- Hyperledger Fabric: PBFT 变种

■ 航空航天

- 飞机控制系统
- 卫星通信
- 需要极高可靠性

■ 金融系统

- 跨境支付
- 证券交易
- 需要防篡改

TIP

拜占庭容错在开放、不可信的环境中特别重要，如公有区块链。而在可信环境中（如企业内部），通常不需要考虑拜占庭故障。

10 FL 不可能定理

FLP 不可能定理由 Fischer、Lynch 和 Paterson 于 1985 年证明。

10.1 定理内容

在异步分布式系统中，即使只有一个进程可能崩溃，也不存在一个确定性算法能够在有限时间内达成一致。

关键假设：

- 异步系统：消息延迟无上限
- 可能发生崩溃故障
- 确定性算法
- 需要在有限时间内终止

10.2 直观理解

为什么不可能：

- | | |
|---|---------------------|
| 1 | 1. 在异步系统中，无法区分： |
| 2 | - 节点很慢 |
| 3 | - 节点已崩溃 |
| 4 | |
| 5 | 2. 如果等待太久，可能永远无法决定 |
| 6 | 3. 如果不等待，可能错过关键信息 |
| 7 | 4. 因此无法保证在有限时间内达成一致 |

10.3 实际意义

理论意义

- 证明了分布式共识的根本困难
- 解释了为什么需要妥协

实践指导

如何应对：

1. 引入随机化：
 - 随机化算法可以绕过 FLP
 - 如 Ben-Or 算法
2. 使用故障检测器：
 - 不完美的故障检测器
 - 最终准确的检测器
3. 接受概率性：
 - 不保证一定成功
 - 但以高概率成功
4. 部分同步模型：
 - 假设最终会同步
 - 如 Raft、Paxos

NOTE

FLP 定理并不意味着分布式共识不可能，而是说在纯异步系统中无法保证确定性地在规定时间内完成。实际系统通过各种方式绕过这个限制。

10.4 与 CAP 的关系

FLP 和 CAP 都揭示了分布式系统的根本限制：

- FLP：异步系统中无法保证一致性 + 可用性 + 终止性
- CAP：分区发生时无法保证一致性 + 可用性

两者都说明：**分布式系统必须做出权衡。**

TIP

理解 FLP 定理有助于我们认识到分布式系统的局限性，从而设计更合理的系统和协议。

11 总结

分布式系统理论为我们提供了理解和设计分布式系统的理论基础：

- CAP 定理：帮助我们理解一致性和可用性的权衡
- BASE 理论：提供了实现高可用性的思路
- 一致性模型：定义了不同强度的一致性保证
- 分布式事务：提供了跨节点保持一致性的方法
- 分布式锁：实现了分布式环境下的互斥访问
- Leader 选举：解决了协调者选择问题
- 拜占庭容错：处理恶意节点的场景
- FLP 定理：揭示了分布式共识的根本限制

本章完